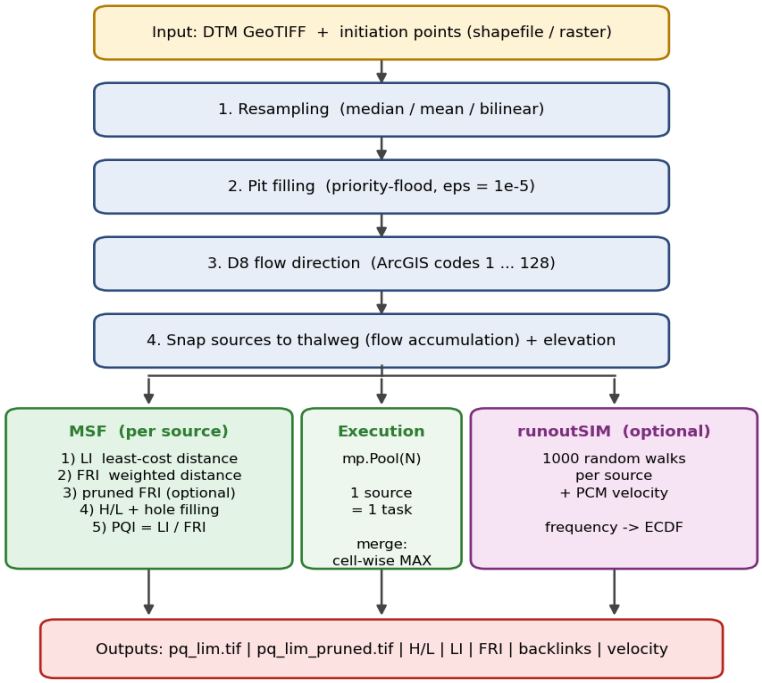
DF-scan

Technical report: how it works, how the path pruning behaves, and a critical review of the code

Modified Single Flow (LI / FRI / PQI) · geomorphological path pruning · runoutSIM random walks with PCM friction

Basis: full reading of **src/dfscan_core.py** (4343 lines) plus targeted reading of the GUI and README, and runs of the real functions on the repository test data (Grigna DTM, 20 m, 9 initiation points).



Executive summary. DF-scan is a regional-scale debris-flow runout tool. For every initiation point it finds the least-cost downslope path (LI), re-weights it by how far it strays from the steepest-descent direction (FRI), keeps cells whose energy line is steep enough (H/L), and reports the ratio LI/FRI (PQ_LIM, always between 1 and 2). The zone has no holes: a cell is kept if it, or anything downstream of it on its path, passes H/L. Results from many sources are merged with a cell-wise maximum. A path-pruning variant stops lateral spreading *during* the search while keeping the steepest-descent line alive. **Verdict:** the approach is sound and the main output is computed correctly; however I found **three real bugs** in how secondary rasters and the sequential mode are assembled (all three reproduced by running the code), plus several design caveats listed in Section 9.

Contents and reading guide

Section	What you will find
1. Overview and outputs	What the tool computes, which rasters it writes, how to read their values
2. Software structure	Files, entry points, dependencies
3. Preprocessing	Resampling, pit filling, D8 flow direction, snapping of sources
4. The MSF engine	LI, FRI, PQI, H/L, hole filling and PQ_LIM, step by step with code excerpts; 4.5 flow continuity when conditions resume downstream; 4.6 which slope threshold controls what
5. Path pruning	Detailed algorithm, how the elimination propagates, why the steepest-descent line survives, measured sensitivity
6. runoutSIM	Stochastic random walk, PCM velocity model, ECDF aggregation
7. Aggregation and parallelism	How per-source results are merged
8. Parameters	Defaults and effect of every important parameter
9. Assessment	What is good, what is wrong (verified vs. inferred), suggested fixes
10. Method and reproducibility	What was read and what was executed

Notation. *LI* = Least-cost (surface) distance Index; *FRI* = Flow Resistance Index (weighted distance); *PQI* = LI/FRI; *PQ_LIM* = PQI restricted to the H/L-compliant zone; *D8* = single-direction steepest-descent flow direction on 8 neighbours; *H/L* = vertical drop over travel length (tangent of the energy-line angle, Fahrböschung). Code references such as r.1648 are line numbers in **dfscan_core.py**.

1. Overview and outputs

DF-scan estimates, at regional scale, **where a debris flow can travel** after it is triggered at known points. The core is the **Modified Single Flow (MSF)** model of Huggel et al. (2003) and Gruber et al. (2009): for each initiation point a minimum-cost path is computed on the DTM, weighted by an index of deviation from the steepest-descent line, and truncated with the H/L ratio. The repository adds two extensions:

- **MSF pruned**: an in-house variant that cuts inefficient lateral branches while the search runs (Section 5).
- **runoutSIM**: a Python port of the R package of the same name – stochastic random walks with a Perla-Cheng-McClung (PCM) friction model, giving per-cell velocity and an empirical probability-like ranking (Section 6).
- An integrated preprocessing chain and a PyQt5 GUI with live log and cartographic preview.

Output raster	Meaning	Range
pq_lim_*.tif	PQ_LIM: path-efficiency index LI/FRI inside the H/L-compliant zone; cell-wise maximum over all sources	1.0 - 2.0
pq_lim_*_pruned.tif	Same, but FRI computed by the pruned search; always a subset of the standard result	1.0 - 2.0
hl_ratio_msf.tif (opt.)	H/L = (source elevation - cell elevation) / LI	0.12 - 0.72 in test
li / fri / backlink (opt.)	Distances and arrival-direction codes (1-8)	metres / code
pq_lim_*_runoutsim*.tif	Empirical CDF of how often random walks cross each cell	0 - 1
runoutsim_max_velocity.tif	Maximum PCM velocity reached in the cell	m/s (test: 0.4 - 18.1)

Important interpretation note. PQ_LIM is a dimensionless *efficiency index* bounded by construction in [1, 2]; the runoutSIM ECDF is a *relative ranking*. Neither is a calibrated probability of occurrence, and the two should not be compared numerically.

2. Software structure

File	Lines	Role
main.py	28	Entry point: no arguments opens the GUI; --config runs the workflow headless
src/dfscan_core.py	4343	The whole engine: Config, raster utilities, preprocessing, MSF (LI, FRI, pruned FRI), runoutSIM, parallelism, main()
src/dfscan_gui.py	2410	PyQt5 tabbed GUI (Inputs, Resampling, Parallel, Processing, MSF, runoutSIM), live log, map preview
msf_config.json, test_data/	-	Saved configuration; test data (grigna_dtm.tif at 10 m, 9 initiation points, example outputs)
DF-scan_Report_Tecnico_*.pdf	6 pp.	Existing report comparing MSF / pruned / runoutSIM on the Valcon-Grigna basin

Dependencies: rasterio, numpy, geopandas (shapefile inputs), PyQt5 (GUI); WhiteboxTools is optional (alternative pit filling and D8). The engine is plain Python with `heapq` priority queues; parallelism is process-based (`multiprocessing.Pool`), one source per task.

3. Preprocessing

3.1 Resampling (`resample_dtm_aggregate`, r.732). Aggregates the DTM to a coarser grid by median, mean or bilinear interpolation (test: 10 m → 20 m, 1224x1034 → 612x517 cells). After resampling, pit filling and D8 are always recomputed. Coarser grids make regional runs affordable but smooth gullies, so the target resolution is a scientific choice, not just a speed knob.

3.2 Pit filling (`fill_pits_custom`, r.1102). An 8-connected priority-flood: start from the raster border and from cells adjacent to NoData (which act as outlets); repeatedly pop the lowest cell and raise each unvisited neighbour to `max(z, z_current + 1e-5)`. The epsilon is meant to leave a tiny gradient across filled areas so that D8 is defined there. WhiteboxTools breach+fill can be used instead.

3.3 D8 flow direction (`calculate_d8_flow_direction`, r.1368). For each cell the neighbour with the largest *drop / distance* is selected (diagonals divided by sqrt 2), using ArcGIS codes (1=E, 2=SE, 4=S, 8=SW, 16=W, 32=NW, 64=N, 128=NE). Cells without a positive slope get 255 (nodata). In the test run: 205,864 cells with a direction and **349 flat/pit cells** after filling. `fdir_to_degrees` converts codes to azimuths (N = 0, clockwise) for the angular tests used later.

3.4 Sources and snapping (main, r.3990-4062). Sources come from a shapefile (rasterized) or a raster (cells ≠ 0). With `SNAP_TRIGGERS` on, a D8 flow accumulation is computed (cells sorted by elevation, descending) and every point is moved, within `SNAP_RADIUS` pixels, to the cell with the highest accumulation, then raised by `ADD_ELEVATION_METERS` (default 1 m). The purpose is to start propagation in the channel instead of on the adjacent hillslope.

4. The MSF engine, step by step

For **each source**, `process_single_source_point` (r.2912) runs: LI → FRI (→ pruned FRI) → HI and H/L → hole filling → PQI → PQ_LIM. Sources are independent; results are merged with a maximum (Section 7).

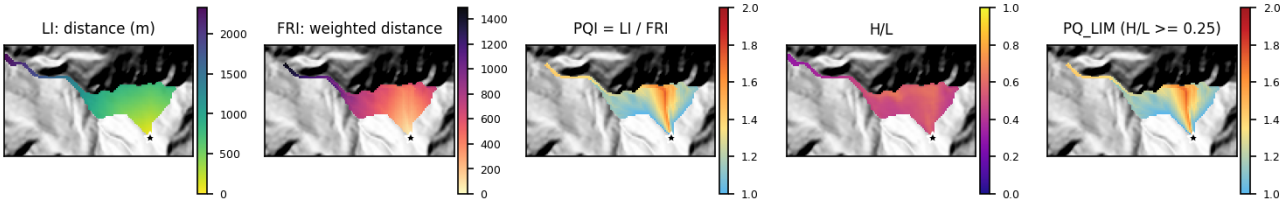


Figure 2 - All stages for one source (S3 of the test, 20 m DTM). LI grows with travel length; FRI is larger where the path deviates; PQI = LI/FRI is highest along the steepest line (red, most probable) and lowest at the fan edges (light blue); PQ_LIM keeps only the H/L-compliant zone. The star marks the initiation cell.

4.1 LI - least-cost surface distance (`path_allocation_li`, r.1648)

Dijkstra on 8 neighbours starting from the source cell (cost 0), accumulating the **surface length** of each step (cellsize, or cellsize x sqrt 2 for diagonals). A move to a neighbour is allowed only if *all* of the following hold:

- **Uphill tolerance**: `dz = z_neighbour - z_current <= tol`, where `tol = round(cellsize x tan(MAX_SLOPE_DEGREES))` in whole metres. The flow can therefore climb small bumps. Computed with the real function: 20 m / 5 deg → 2 m (exact 1.75); 5 m / 30 deg → 3 m (2.89); 20 m / 30 deg → 12 m (11.55).
- **Angle vs. D8 of the departure cell**: `|move_angle - fdir_from| < HRMA_FROM_THRESH_LI` (default 90 deg): the flow may not move "against" the local drainage direction.
- **Angle vs. D8 of the arrival cell**: same test with `HRMA_TO_THRESH_LI`. Cells without D8 (NaN) do not constrain the move.
- The neighbour is updated if `new_dist < dist[neighbour]`; the code also stores a **backlink** (code 1-8 of the move) and **start_z** (source elevation, propagated along the path).

The +/-90 degree cone of LI is deliberately wide: LI answers "how long is the shortest *physically admissible* route to this cell?", without penalising deviation yet. Note that LI does **not** stop at an H/L limit; H/L is applied afterwards (4.4).

4.2 FRI - deviation-weighted distance (`path_allocation_fri`, r.1890)

A second Dijkstra with the same neighbourhood and uphill tolerance, but (i) cells without a D8 direction are excluded, and (ii) every step is weighted:

```
angle_from = |move - fdir(current)|          # folded into [0, 180]
angle_to   = |move - fdir(neighbour)|
if angle_from > CUT_ANGLE or angle_to > CUT_ANGLE:  move forbidden      # default 45 deg
avg_angle  = (angle_from + angle_to) / 2
hf         = ZERO_FACTOR + avg_angle * SLOPE                            # 0.5 + avg * 0.011111
cost       = step_length * hf ;   FRI(neighbour) = FRI(current) + cost
```

With the defaults, hf ranges from 0.5 (perfectly aligned with D8) to 1.0 (45 degrees off). FRI is therefore between half and one times the path length. The hard cut at 45 degrees is what prevents the flow from running sideways across slopes in the standard model.

4.3 PQI - path-efficiency index

`PQI = LI / FRI` (r.3086). For a uniform path with mean deviation *t* degrees, PQI = 1 / (0.5 + 0.011111 t): exactly **2.0** on the steepest-descent line and **1.0** at 45 degrees of deviation. **Worked example:** a 1000 m route whose steps deviate 19 degrees on average has FRI = 1000 x (0.5 + 0.211) = 711 m and PQI = 1.41. Figure 3 relates pruning thresholds to the mean deviation they allow.

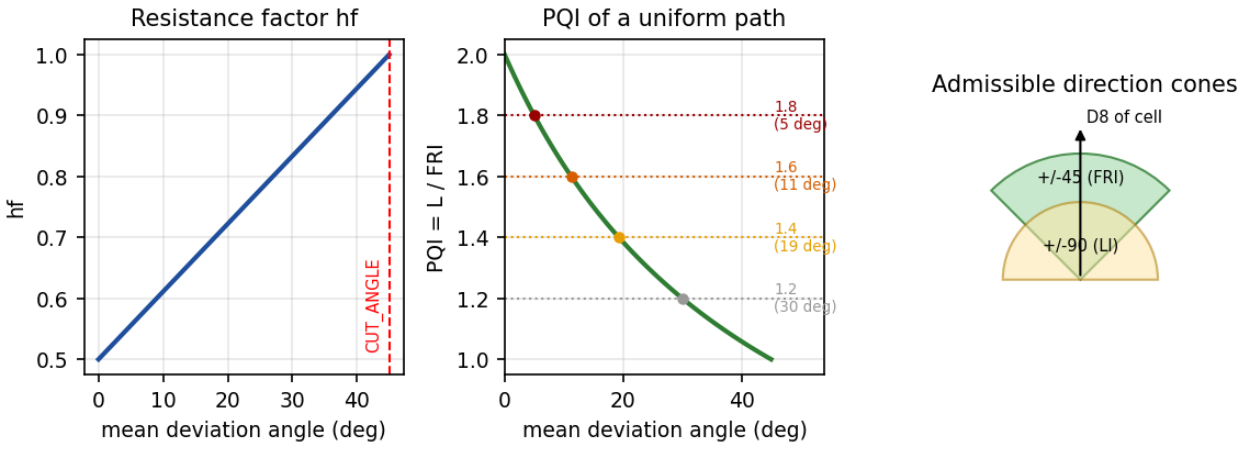


Figure 3 - Left: resistance factor hf versus deviation. Centre: PQI of a uniform path, with the mean deviation corresponding to each pruning threshold (1.2 = 28 deg, 1.4 = 19, 1.6 = 11, 1.8 = 5). Right: admissibility cones (LI +/-90, FRI +/-45).

4.4 H/L, hole filling and PQ_LIM

- **HI** = `start_z - z_cell` (drop from the source). **H/L = HI / LI** (path distance) or **HI / Euclidean distance** if `USE_DIRECT_DISTANCE_FOR_HL` is set.
- **Stopping mask**: cells with `H/L >= H_L_THRESHOLD` are kept (0.25 in the code; the README says 0.19).

- **Hole filling** (`FILL_HL_HOLES`, r.3046-3074): cells are sorted by decreasing LI; whenever a cell is kept, its parent (via the backlink: `parent = cell - move`) is kept too. This guarantees that the propagation zone is **connected to the source** even where H/L locally dips below the threshold. Visible side effect: 46% of cells in the test H/L raster are below 0.25 although they belong to the zone – they are the connecting tissue.
- **PQ_LIM = PQI** only in kept cells where FRI is defined (> 0).

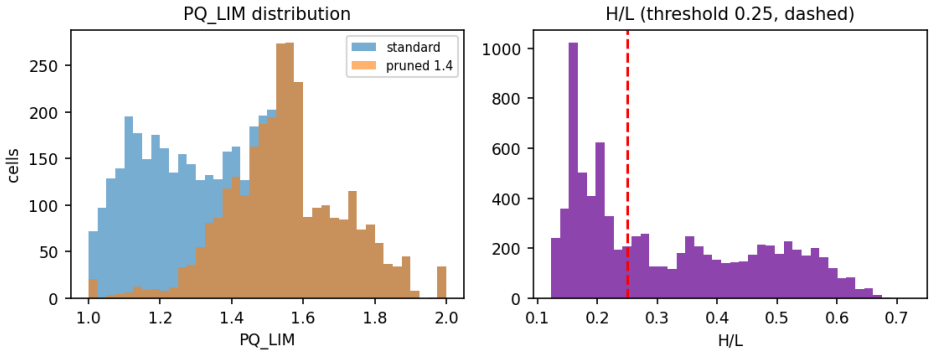


Figure 4 - Test data: distribution of PQ_LIM (standard vs. pruned 1.4) and of H/L. Pruning removes mostly low-PQI cells and shifts the mean PQ_LIM from 1.41 to 1.55.

4.5 Flow continuity: what happens when the conditions resume downstream

A real debris flow can lose energy on a gentle reach and, if it is still moving, regain it on a steeper reach further down. How the two models treat this is a key design difference. All numbers below were obtained by running the real functions on a **synthetic channel** (steep 31 deg for 800 m, gentle 1.1 deg for 2200 m with a 3 m counter-slope bump, steep 35 deg for 1000 m, gentle 1.7 deg for 1200 m; V-shaped valley, 20 m cells) so that the expected behaviour is known in advance.

- **H/L is cumulative.** H/L = (z_0 - z) / L is measured from the source, not locally. On a gentle reach it decays (the energy line rises above the terrain), but when a steep reach follows, the drop grows faster than the length and H/L **recovers**. In the synthetic channel H/L falls to 0.17 on the gentle reach and climbs back to 0.31 at the end of the second steep reach (Figure 5). The model therefore "resumes" naturally: the same cells that fail H/L upstream are followed by cells that pass it downstream.
- **Hole filling gives the zone continuity (see the box below).** Without it the channel cells kept by H/L alone are 175 of 260 and the zone is cut in two; with it 250 of 260 are kept: 75 cells of the gentle reach are bridged because they are the LI-parents of kept downstream cells. In the real test data it adds 23, 5 and 68 cells to sources S4, S5 and S6 and joins the two separate pieces that S5 and S6 would otherwise have (Figure 7, right; Figure 6a).
- **Physical caveat.** The bridging is a modelling choice, not physics: a flow that actually stopped on a long flat reach would not restart. Hole filling assumes that if *any* downstream cell is reachable with enough energy, the whole connecting path was travelled. It is conservative (it over-predicts) and can bridge arbitrarily long gaps; there is no maximum gap length or minimum slope.
- **Small obstacles.** A counter-slope inside the channel is first removed by the pit filling (7 cells raised in the synthetic channel), so the uphill tolerance never sees it: tolerance 0 m and 2 m gave identical results. The tolerance therefore matters for lateral and diagonal moves and for non-fillable roughness, not for dams in the thalweg.
- **Where continuity can still break.** (i) FRI cannot cross cells without a D8 direction (flats and pits that survive filling) and cannot make moves deviating more than CUT_ANGLE from the local D8, while LI can. In the 9-source test, 293 of 5198 H/L-compliant cells (5.6%) have no FRI and therefore do not appear in PQ_LIM; raising CUT_ANGLE to 180 degrees reduces them to 81, so about 210 are lost to the 45 degree cut and the rest to flats/upstream barriers. (ii) The pit-fill epsilon can fail at altitude (defect 5 in Section 9).
- **runoutSIM does not resume.** Its continuity comes from momentum, not from a cumulative energy line. In the same synthetic channel, 300 walks all cross the first steep reach (100% reach column 40) but the farthest walk gets only 2 cells (mu = 0.06, farthest column 42) to 17 cells (mu = 0.02, column 57), i.e. 40-340 m, into

the gentle reach and none reaches the second steep reach, for any mu in 0.02-0.15 with M/D = 45 m. Once v^2 becomes negative the walk ends for good.

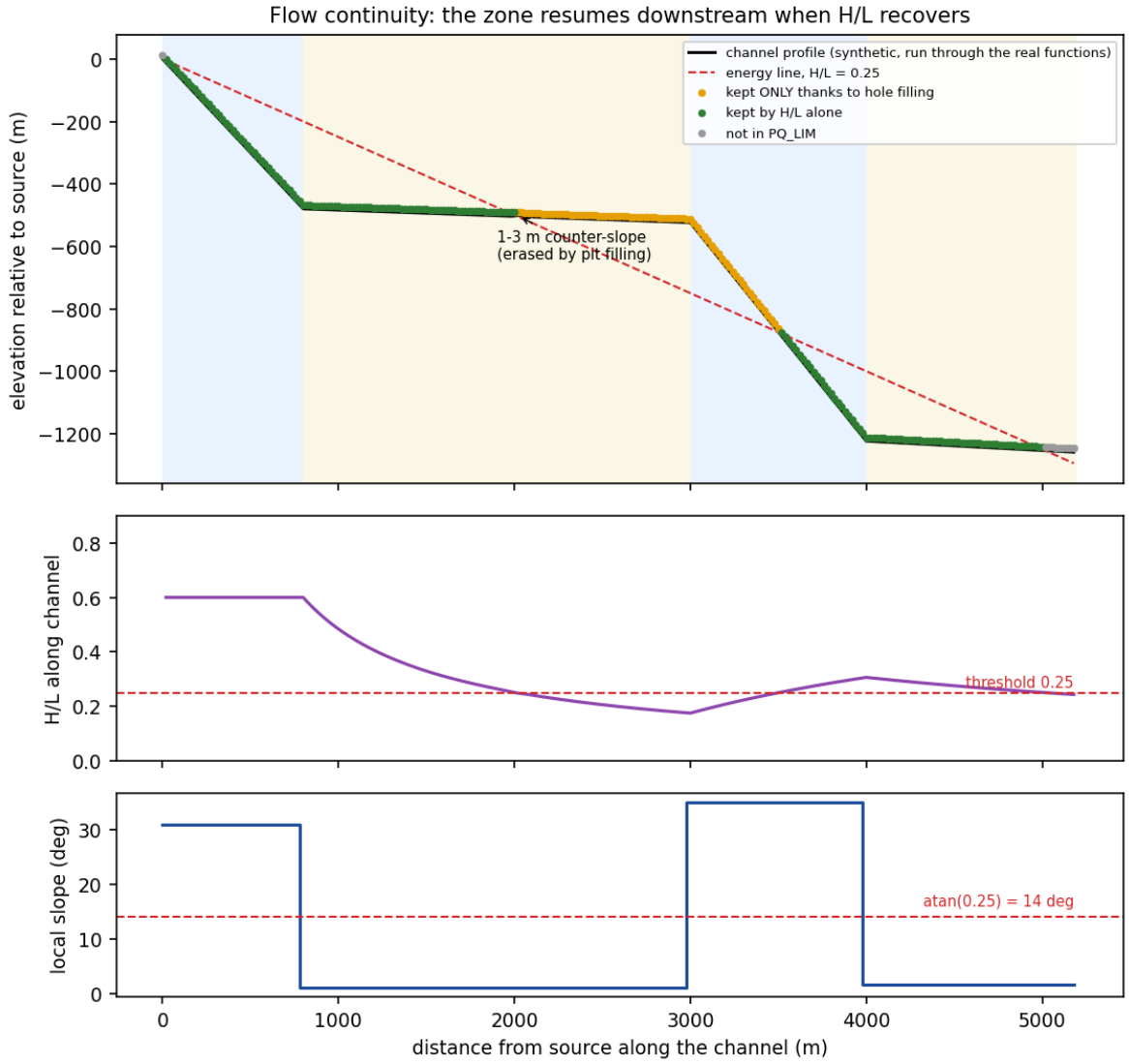


Figure 5 - Continuity on a synthetic channel, computed with process_single_source_point. Top: profile, H/L = 0.25 energy line and per-cell status (green = kept by H/L alone, orange = kept only thanks to hole filling, grey = excluded). Middle: H/L along the channel dips below the threshold on the gentle reach and recovers on the second steep reach. Bottom: local slope against the 14 degree slope equivalent to H/L = 0.25. Beyond about 5000 m H/L stays below the threshold and the zone ends, as it should.

A feature worth highlighting: the zone has no holes. The energy-line criterion is evaluated at the *end* of the run-out, not along it. A cell is kept if it passes H/L itself **or if any cell downstream of it on the same least-cost path does**. So when conditions weaken on a gentle reach and recover further down, before the final H/L stop, the weak stretch is not dropped: the zone stays continuous from the source to the farthest compliant cell, and the cells in between receive their own PQI value.

How it works in the code (`process_single_source_point`, r.3046-3074). Cells are visited from the farthest to the nearest (LI sorted in descending order) and the "keep" flag is passed from each kept cell to its LI parent through the backlink. Because children are always processed before their parents, one pass propagates the flag along a chain of any length (cost O(N log N) for the sort, O(N) for the loop):

```
keep  = (h_l != nodata) & (h_l >= H_L_THRESHOLD)          # cells that pass the energy line
order = valid LI cells sorted by LI DESCENDING             # farthest first
for (r, c) in order:
    b = backlink_li[r, c]                                  # code 1..8 of the move that reached (r, c)
    parent = (r, c) - offset[b]                            # the cell it came from
    if keep[r, c]:
        keep[parent] = True                                # child kept  ->  parent kept
```

Two continuity mechanisms exist, and they act on different quantities. The standard model has *no* threshold on PQI (the index is only reported); the only threshold is H/L. In the pruned model a second, PQI-based, mechanism appears:

	H/L continuity (standard and pruned)	PQI continuity (pruned only)
Quantity tested	Cumulative drop over travel length vs. H_L_THRESHOLD	Efficiency LI(neighbour) / d vs. MSF_PRUNING_THRESHOLD
If a cell dips below	It is kept if **any descendant** on its LI path passes: look-ahead, so a dip that recovers downstream is bridged	Lateral edges are cut at once (no recovery); the **D8 edge is never cut**, so the main line goes on and its PQI can recover when the path straightens
Measured (test data)	S6 +68 cells, S4 +23, S5 +5; two split zones rejoined (Figure 6a, Figure 7 right)	Threshold 1.4: 45 D8-line cells below threshold kept, 28 later back above it (5 by at least 0.05). Threshold 1.6: 284 kept, 36 back above (21 by at least 0.05) (Figure 6b-c)
Limits	No maximum gap length; cells without FRI remain holes in PQ_LIM; follows the LI tree, not the FRI tree	Recovery is typically modest; no look-ahead for lateral branches, so a cut lateral branch is never reconsidered
Result	Zone continuous up to the farthest compliant cell	Main flow line never interrupted by the efficiency threshold

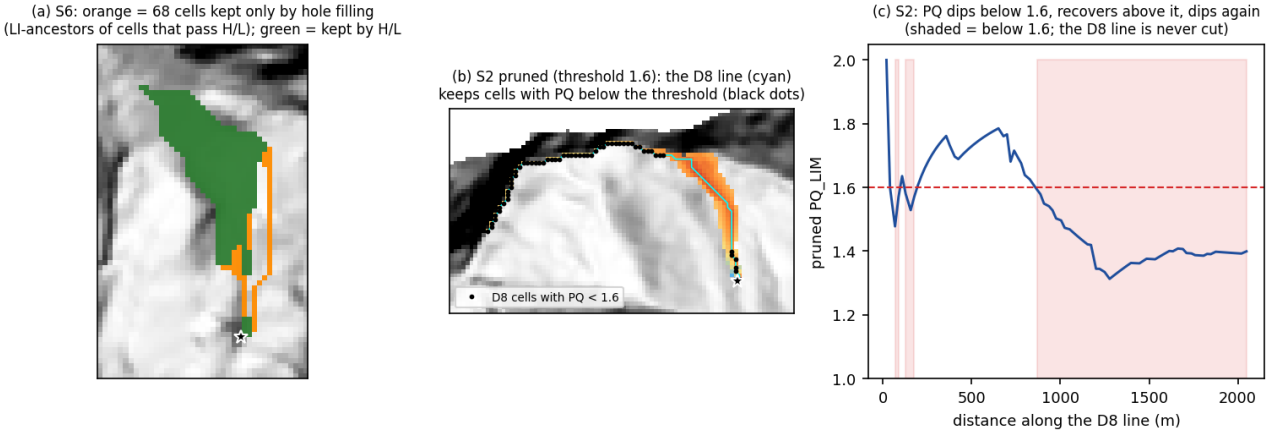


Figure 6 - Continuity on real data. (a) Source S6: orange cells are kept only because they are LI-ancestors of cells that pass H/L. (b) Source S2 with pruning threshold 1.6: black dots are D8-line cells whose PQ is below the threshold yet are kept. (c) PQ along the same D8 line: it drops below 1.6, recovers above it between 200 and 870 m, then falls below again.

Reading Figure 6 honestly. The H/L bridging is robust and always on. The PQI behaviour is real but weaker than the H/L one: along the D8 lines the efficiency often stays below the threshold after a dip (S2 ends at 1.40, below 1.6), so a dip does not always reverse. What the model guarantees is that the line is *not cut*; whether PQI recovers depends on the terrain.

4.6 Slope thresholds: which parameter controls which slope

"Slope threshold" can mean several different things in DF-scan. They are easy to confuse, so they are separated here:

Parameter	What it really thresholds	Equivalent value / effect
MAX_SLOPE_DEGREES	Allowed **uphill step** between adjacent cells, in LI and FRI (not a limit on the flow slope)	tol = round(cellsize x tan(angle)): 0.5 deg → 0 m; 5 deg → 2 m; 30 deg → 12 m (20 m cells). Sweeping 0.5-30 deg changes PQ_LIM by only +2.6% (4900 → 5029 cells) and LI by +12% on the test data (Figure 7, left)
H_L_THRESHOLD	The **reach (energy-line) slope** from source to cell: the actual stopping criterion	tan(angle): 0.25 → 14.0 deg; 0.19 → 10.8 deg. Very sensitive: 0.15 → 7691 cells, 0.25 → 5001, 0.45 → 2382 (Figure 7, centre)
CUT_ANGLE, HRMA_*	**Direction** deviation from D8 (degrees), not slope	FRI: 45 deg; LI: 90 deg

Parameter	What it really thresholds	Equivalent value / effect
RUNOUTSIM_SLOPE_THRESH	Neighbour slope above which the walk is **forced to the steepest neighbour** (gamma > 1)	40 deg; below it the choice is stochastic. It never stops a walk
RUNOUTSIM_FRICTION (mu)	Slope at which velocity stops increasing: velocity grows only where tan(slope) > mu	mu = 0.06 → 3.4 deg; memory of the entry velocity decays as exp(-L / (M/D)): 45 m for M/D = 45, v^2 halves every ~16 m
(none)	There is **no minimum slope** in MSF: a cell is stopped only by H/L, by the 45 degree cut or by the absence of D8	Flat reaches are crossed by LI, but FRI cannot cross cells without D8

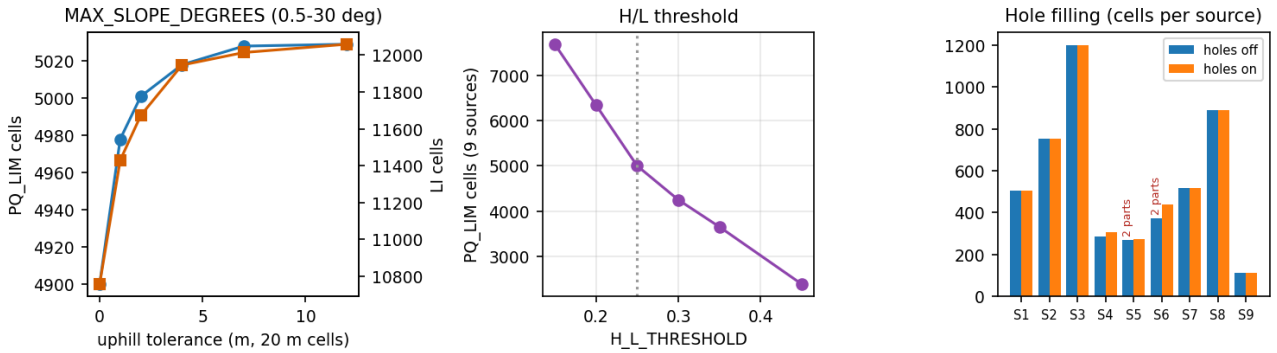


Figure 7 - Left: sensitivity to the uphill tolerance (MAX_SLOPE_DEGREES), weak on this DTM. Centre: sensitivity to H_L_THRESHOLD, strong. Right: cells per source with and without hole filling; red labels mark sources whose zone splits into disconnected parts without it.

Practical consequence. The parameter that governs where the flow stops is H_L_THRESHOLD, not MAX_SLOPE_DEGREES. The latter is named like a slope limit but only changes how many small obstacles can be climbed. The README description of both parameters should be corrected (defect 8).

5. Path pruning (pruned MSF)

The problem. With an 8-neighbour search and a +/-45 degree cone, the standard FRI lets the flow fan out: every reached cell opens new lateral neighbours, so the footprint widens (test source S3: 1202 cells, first panel of Figure 9). **The idea.** Instead of filtering the finished raster, **stop expanding lateral branches** whose efficiency drops below a threshold, while **always leaving the steepest-descent branch free**.

5.1 The algorithm, line by line (`path_allocation_fri_pruned`, r.2140)

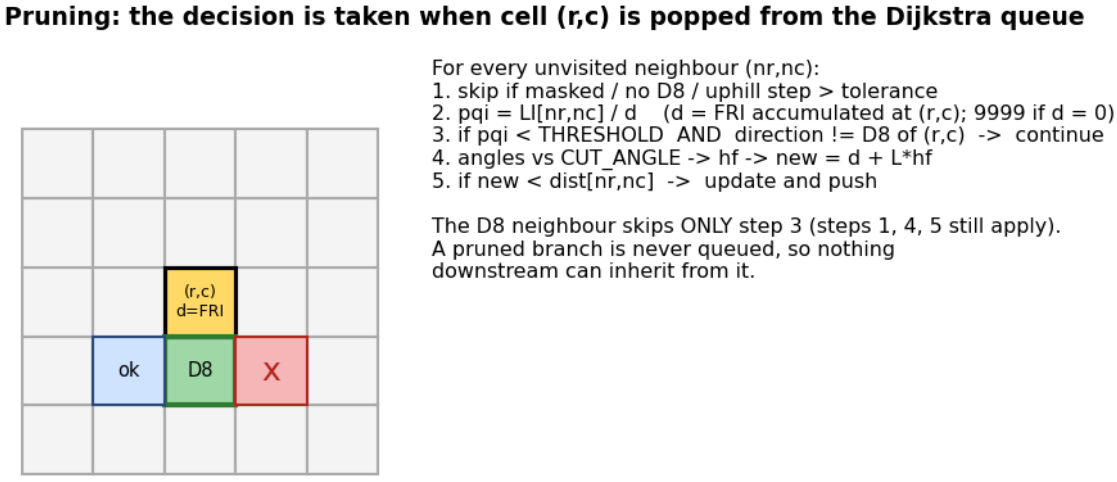


Figure 8 - Decision logic applied to each neighbour of the cell just extracted from the queue.

```
code   = fdir[r, c]                    # D8 code of the cell being expanded (64, 128, 1, ...)
```

```
d8_idx = d8_to_idx.get(code, -1)           # index 0..7 of the steepest neighbour, -1 if no D8
for dir_idx in range(8):
    ... skip if visited / masked / no-D8 / uphill step > tolerance
    l_val = li[nr, nc]                     # LI of the neighbour, from the UNPRUNED LI run
    pqi   = l_val / d if d > 0 else 9999   # d = FRI accumulated at the current cell
    if pqi < pruning_threshold and dir_idx != d8_idx:
        continue                           # branch cut: never queued, never expanded
    ... CUT_ANGLE test, hf, new_dist = d + L*hf, standard Dijkstra update
```

Ingredients. The pruning reuses three things already available: the **LI** raster (complete, unpruned), the **FRI of the current cell** (d) and the **D8 direction of the current cell**. For each neighbour it estimates the efficiency the path would have there, `LI(neighbour) / d`; if that is below the threshold *and* the neighbour is not the steepest-descent direction, the edge is discarded.

5.2 How the elimination propagates

- **The decision is on the edge, not on the cell.** A neighbour discarded from one cell can still be reached from another already-extracted cell that evaluates it favourably. A cell disappears only if *every* incoming edge is cut.
- **A cut branch is never pushed into the queue**, so it receives no FRI and generates no children: everything that depends only on it vanishes. This is what distinguishes pruning from a final filter, which would leave orphan cells and holes in the middle of the zone.
- **The denominator grows along lateral branches.** As a branch moves away from the axis, hf rises towards 1.0 and d grows faster than LI, so efficiency decays and eventually crosses the threshold. High thresholds (1.6-1.8) cut after very few lateral steps; low ones (1.2) allow mean deviations up to ~28 degrees.
- **The source is never pruned**: at `d = 0` the code uses `pqi = 9999`, so the first moves are free (the initial fan forms, then gets narrowed).

5.3 What "the steepest descent remains" really means

The exemption applies to the D8 direction **of every extracted cell**, not only to the main channel of the initiation point: the test `dir_idx != d8_idx` skips only the threshold check (step 3 of Figure 8). Uphill tolerance, CUT_ANGLE, the D8 mask (flat cells with no D8 can never be crossed) and the Dijkstra comparison remain active. Two consequences:

- The main channel can never be interrupted by the threshold: in Figure 9 the cyan line always stays inside the kept zone, even at 1.8 where only 46 cells survive against 1202.
- Every lateral cell that *passes* the threshold drags its own downslope D8 path with it, without further efficiency checks. For that reason high thresholds produce thin tongues rather than a compact core.

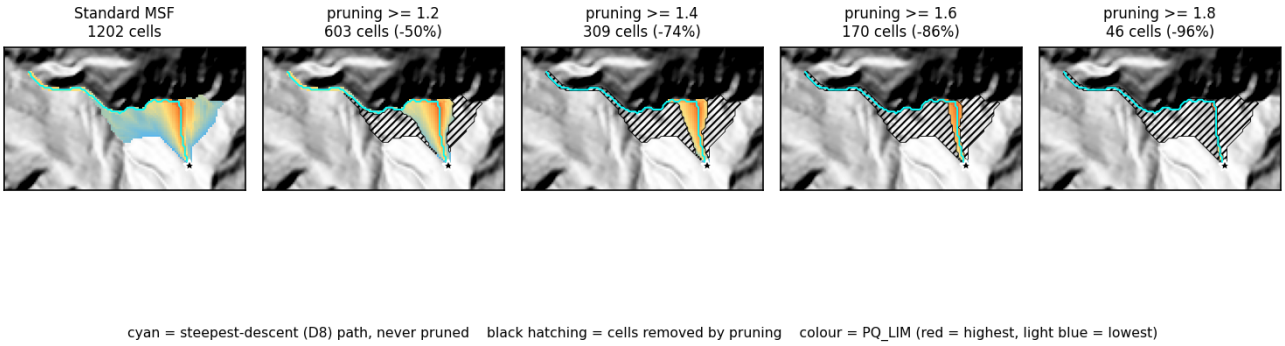


Figure 9 - Source S3: standard MSF versus pruned MSF with thresholds 1.2-1.8, computed by the real process_single_source_point. Black hatching = cells removed by pruning; cyan = steepest-descent (D8) path; colour = PQ_LIM (red = highest, light blue = lowest). Note how the cyan line is never hatched.

5.4 Measured sensitivity to the threshold

Threshold	Equivalent mean deviation	Cells (sum of 9 sources)	% of standard	Kept cells with PQI < threshold
standard	-	5001	100%	-
1.0	45 deg (no cut)	5001	100%	0%
1.2	28 deg	3905	78%	7.8%
1.4	19 deg	3106	62%	17.2%
1.6	11 deg	1865	37%	49.5%
1.8	5 deg	1146	23%	81.0%

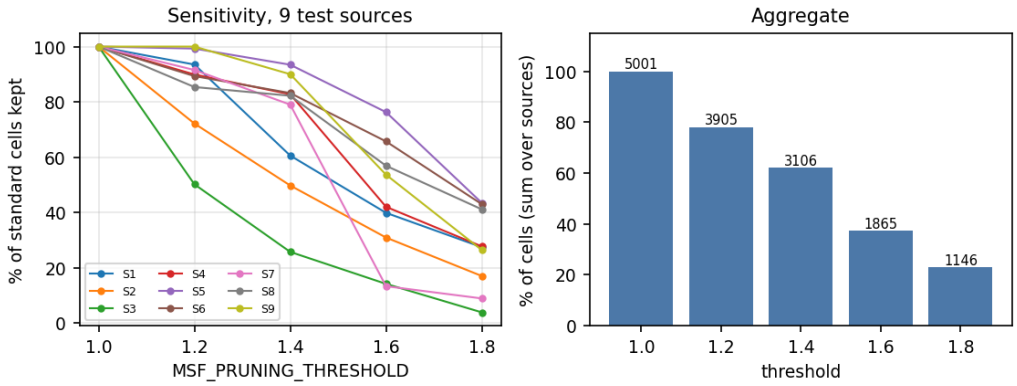


Figure 10 - Fraction of cells kept as the threshold increases, for each of the 9 test sources (left) and aggregated (right). Sources differ markedly: S3 loses 74% at 1.4 while S5 loses only 7%.

Reading the threshold correctly. It is **not a hard filter on the final PQ_LIM**: in the pruned test raster (threshold 1.4) 16.6% of the cells still have PQ_LIM below 1.4 and the minimum is 1.0. From the code: (a) the test uses d of the current cell rather than `new_dist`, so it is optimistic by about one step; (b) the D8 exemption lets low-efficiency cells through; (c) the final PQ_LIM is recomputed as `LI / FRI_pruned` of the cell, which can differ from the estimate used during pruning. The threshold should therefore be calibrated empirically (test case: 1.2 → -22% cells, 1.4 → -38%).

5.5 Comparison with alternatives

Approach	Advantages	Drawbacks
Post-hoc filter (delete PQ_LIM < s)	Trivial	Leaves disconnected cells; paths unchanged; can cut the channel
In-search pruning (DF-scan)	Keeps connectivity; cheaper (pruned branches are never expanded); channel protected; a single parameter	Threshold is not a bound on the final value; depends on unpruned LI and on D8
Narrower CUT_ANGLE only	Simple	Cuts uniformly, also the legitimate early fan and real channel bends
runoutSIM	Physical (velocity, inertia); empirical probability	Stochastic and slower; friction parameters need calibration

6. runoutSIM: random walks and the PCM model

For each source (`runout_sim_walks`, r.2442) the code launches `RUNOUTSIM_WALKS` (default 1000) random walks. At every step, from the current cell only the strictly *lower* neighbours are considered:

- **Slope and gamma**: beta = atan(dh / dist); `gamma_i = tan(beta_i) / tan(RUNOUTSIM_SLOPE_THRESH)` (threshold 40 deg).
- **Persistence**: the neighbour in the direction of the previous step has its weight multiplied by PERSISTENCE (1.2-1.6), so the flow tends to keep going straight.

- **If gamma_max > 1** (slope beyond the threshold) the step is **forced to the steepest neighbour** (random tie-break).
- **Otherwise** the candidates are the neighbours with `gamma_i >= gamma_max^E_DIV`, drawn with probability proportional to persistence x tan(beta). A large E_DIV (3.5) shrinks the candidate set and thus the divergence; a small one widens it.
- **PCM velocity** (`pcm`, r.2398): alpha = g (sin t - mu cos t); beta = -2 L / (M/D); v^2 = alpha (M/D)(1 - e^{beta}) + v_prev^2 e^{beta} cos(delta_t), with the concave-transition correction of Wichmann (2017). A walk stops when v^2 < 0 (the mass halts), when no lower neighbour exists, or at the raster edge / NoData.

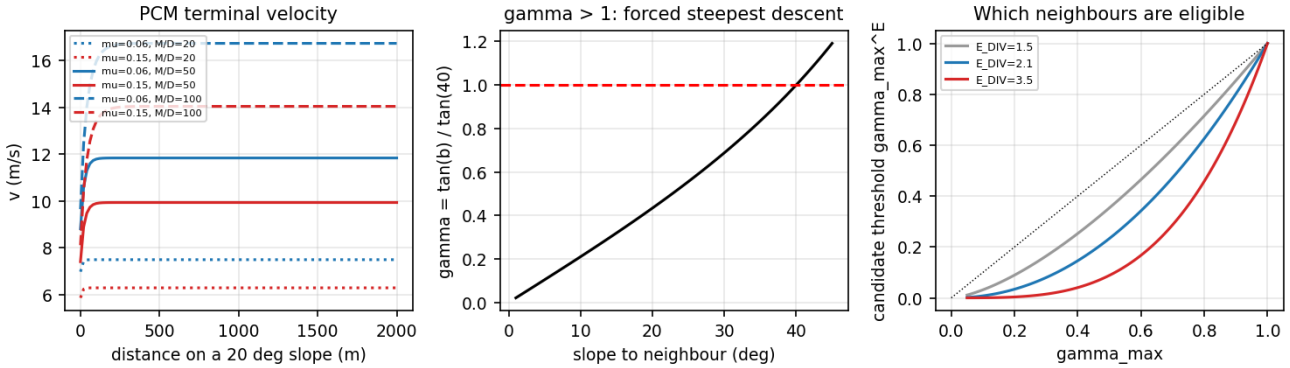


Figure 11 - Left: PCM velocity on a constant 20 degree slope for different friction mu and mass-to-drag ratio M/D (computed with core.pcm): the velocity saturates towards a terminal value. Centre: gamma and the forced-steepest-descent rule. Right: eligibility threshold for neighbours as a function of E_DIV.

Aggregation. Each cell counts in how many walks it was visited (once per walk). Frequencies from different sources are summed (optionally weighted by a source probability raster) and the final map is the **ECDF** (`calculate_ecdf`, r.2858): for each cell, the fraction of reached cells with frequency <= its own. It is thus a **relative ranking**, not a probability of occurrence. Optionally, connectivity to a feature (road, village) is reported as the fraction of walks that intersect it.

7. Aggregation, parallelism and output

- **Parallel mode** (`process_sources_parallel`, r.3219): `mp.Pool(NUM_WORKERS)`, one source per task, results consumed with `imap` and merged in the main process. Worker logs are routed to the GUI through a queue.
- **Sequential batch mode** (`process_source_batch`, r.3483; BATCH_SIZE = 200): each batch accumulates its own maxima, then `combine_batch_results` merges the batches.
- **Merge rule**: PQ_LIM and pruned PQ_LIM take the **maximum** cell by cell across sources (OVERLAP_METHOD = MAX): a cell is as susceptible as the most efficient path that reaches it.
- **Measured speed** on the test data: 0.2-0.4 s per source at 20 m (612x517 cells) for LI + FRI + pruned FRI + H/L.

8. Parameters

Parameter	Config default	Effect
MAX_SLOPE_DEGREES	30	Uphill tolerance = round(cellsize x tan(angle)) metres
H_L_THRESHOLD	0.25 (README: 0.19)	Energy-line stopping threshold
FILL_HL_HOLES	True	Keeps the kept zone connected to the source
HRMA_FROM/TO_THRESH_LI	90 / 90	Angular cone admitted by LI
ZERO_FACTOR, SLOPE	0.5, 0.011111	hf = zero + angle x slope; maximum PQI = 1 / zero = 2
CUT_ANGLE	45	Maximum deviation admitted in FRI

Parameter	Config default	Effect
ENABLE_MSF_PRUNING / THRESHOLD	False / 1.5	Turns pruning on; threshold on LI/FRI efficiency
USE_DIRECTION_AWARE_UPHILL	False	If True the tolerance uses the real step length (diagonals more permissive)
USE_DIRECT_DISTANCE_FOR_HL	False	H/L with Euclidean instead of path distance
SNAP_TRIGGERS / SNAP_RADIUS	True / 2	Move sources onto the thalweg
RUNOUTSIM_*	E_DIV 2.1, PERS. 1.6, SLOPE 40, WALKS 1000, mu 0.06, M/D 45	Random-walk and PCM parameters

9. Assessment: results, strengths and defects

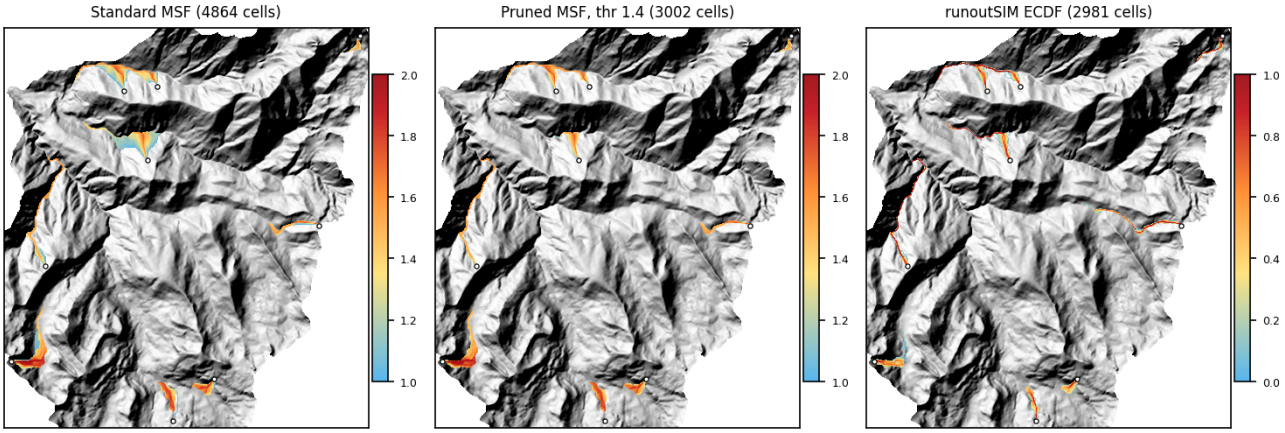


Figure 12 - Repository outputs (test_data/outputs, not regenerated) on the 20 m Grigna DTM with 9 sources: standard MSF 4864 cells, pruned MSF (threshold 1.4) 3002 cells, all contained in the standard result; runoutSIM 2981 cells.

9.1 Zoomed application examples

Four sources from the test set, each shown with the three models (standard MSF, pruned MSF 1.4, runoutSIM ECDF) on the same window. The counts are the cells inside each window.

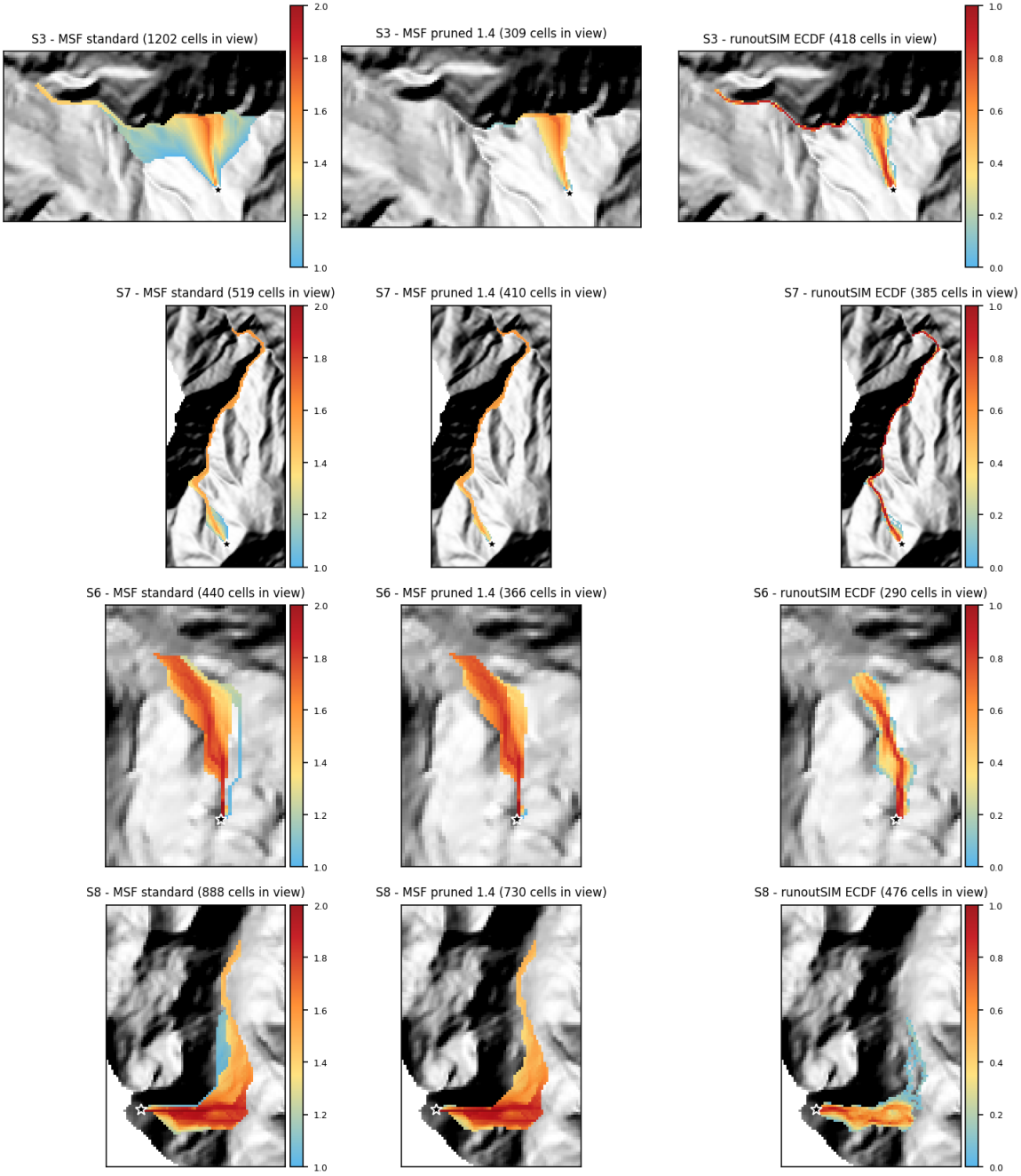


Figure 13 - Zoomed views of S3, S7, S6 and S8 (repository outputs). S3: the wide standard fan (1202 cells) collapses to a tongue along the steepest line (309). S7: the pruned result keeps the whole long channel (410 of 519 cells) and trims only the fan near the source. S6: pruning removes the thin low-PQI strip on the east side (366 of 440), leaving the compact core. S8: pruning removes the low-PQI lobe inside the fan (730 of 888); runoutSIM (476) concentrates on the same channel and shows a side spread to the east.

9.2 What I like

Overall I like the approach and most of the implementation.

- **Clear structure**: LI, FRI and H/L are separate, reusable functions that follow the literature; parameters are few and interpretable.
- **Continuity by construction**: hole filling through the backlinks answers the connectivity problem after H/L truncation in one cheap pass and without a second search; in pruned mode the D8 exemption adds a second, PQI-based continuity of the main flow line (Section 4.5). I consider this one of the best ideas in the code.
- **Pruning inside the Dijkstra loop**: conceptually stronger than a post-filter, with one parameter and a guarantee on the main channel (Section 5). The pruned result is a true subset of the standard one (verified: 3002 of 3002 cells).

- **MAX merging and per-source parallelism**: simple to reason about and scalable with the number of sources; the capture of worker logs for the GUI is well done.
- **Internal consistency**: all outputs share grid and nodata; observed ranges match theory (PQ_LIM in [1, 2], ECDF in [0, 1]).

9.3 Defects and weak points

The main PQ_LIM raster in parallel mode is computed correctly (cell-wise maximum). The defects below concern secondary rasters and the sequential mode. Only items marked **BUG** are errors in the strict sense (wrong or missing output); the others are weaknesses, design notes or documentation issues. Tags: **[V]** = verified by running the real code; **[L]** = inferred from reading, not executed.

#	Where	Problem	Effect	Class / status
1	process_source_batch r.3662-3671	The mask `result.pq_lim > combined_pq_lim` is evaluated **after** combined_pq_lim was already updated with the maximum, so it is always false. Same for FRI (r.3670).	In sequential mode **backlink_li is empty** (0 valid cells over 3 sources) and backlink_fri almost empty (3 cells); H/L comes from the first source that reaches a cell.	**BUG** [V]
2	combine_batch_results r.3796	The returned dictionary omits `pq_lim_pruned`.	With ENABLE_PARALLEL_PROCESSING = False the pruned raster is **never** written; a misleading warning "no pruned cells found (check fdir input)" appears.	**BUG** [V]
3	process_sources_parallel r.3442-3460	LI, FRI and H/L use the same "new pq_lim > combined" condition, evaluated after the combined array has been updated.	In parallel mode the first source reaching a cell wins, not the one giving the maximum PQ_LIM. Run on the 9 test sources: PQ_LIM equals the cell-wise maximum (4864 cells, correct), but on the 137 cells covered by two or more sources LI equals the first source in 137/137 cases and the max-PQ source in only 117/137.	**BUG** [V]
4	calculate_uphill_tolerance r.686	Double rounding (to 0.1, then to an integer, with round-half-to-even). Example: 2.5 → 2; 11.55 → 12.	The tolerance is a non-linear function of the angle parameter (2.89 m becomes 3 m; 1.75 m becomes 2 m).	Quirk, mostly harmless [V]
5	fill_pits_custom r.1170	Epsilon 1e-5 on a float32 array: above ~1000 m the increment is below numerical resolution (~1e-4).	Flat cells remain. Verified on the synthetic channel: at a base elevation of 300 m 1 cell has no D8 after filling, at 1500 m and 2500 m 8 cells do (the 7 raised cells stay flat). The 349 no-D8 cells of the real test DTM are consistent with this but the cause was not isolated there. FRI cannot cross such cells.	Weakness [V] on synthetic
6	Pruning r.2278-2286	Efficiency estimate uses d of the current cell (not new_dist) and unpruned LI; D8 exemption on every cell.	The threshold is not a bound on final PQ_LIM: at 1.4, 16.6% of kept cells have PQ_LIM < 1.4.	Design note, to document [V]
7	runout_sim_walks r.2516	Random generator created without a seed.	Not an error: random walks are stochastic by nature. But with no seed option a run cannot be repeated exactly. Measured on S3 with two runs: 1000 walks give frequency correlation 0.997 (max difference in visit fraction 0.06); 5000 walks 0.9996 (0.02).	Missing option, not a bug [V]
8	README vs. code	README calls H/L a "stopping condition" and MAX_SLOPE a "flow-initiation threshold"; in code H/L is a final filter and MAX_SLOPE is the uphill tolerance. README default H/L 0.19, code 0.25.	LI explores the entire reachable raster for every source, so the per-source cost is not bounded by H/L (test: LI reaches 11,673 cells vs 5,001 in PQ_LIM, about 2.3 times more).	Documentation + performance note [V]

#	Where	Problem	Effect	Class / status
9	PQ_LIM / ECDF	PQ_LIM is an index in [1, 2] and the runoutSIM ECDF a relative ranking; both are called "probability" in the documentation.	Not calibrated, not mutually comparable; risk of misinterpretation in hazard products.	Naming, not an error

9.4 Recommended fixes, in priority order

- Fix 1-3 by updating LI / FRI / H/L / backlinks **before** updating the combined PQ_LIM (or by storing the mask "new pq > old pq" before taking the maximum), and add `pq_lim_pruned` to the dictionary returned by combine_batch_results.
- Add an automated test comparing sequential and parallel modes on the same input: it would have caught 1-3 immediately, and the test data already exist in the repository.
- Expose a random seed for runoutSIM (per source / per worker) in Config.
- Performance: LI and FRI are pure-Python Dijkstra over the whole raster. Apply the H/L cut during exploration (as the README suggests), consider numba, and ship the DTM/D8 to workers with a pool initializer or shared memory instead of including them in the partial passed with each task [L, to be profiled].
- Document the threshold semantics of the pruning (Section 5.4) and rename PQ_LIM so that it is not read as a probability.

10. Method and reproducibility

Read: the README, the configuration, LI / FRI / pruned FRI, process_single_source_point, runoutSIM and PCM, the parallel and batch drivers and main(), plus the details of fill / D8 / snapping. **Executed** (repository untouched): custom pit filling and D8 on the 20 m DTM; process_single_source_point for the 9 test sources at thresholds 1.0-1.8; process_source_batch and combine_batch_results on 3 sources (defects 1-2); process_sources_parallel with 2 workers on the 9 sources compared against single-source runs (defect 3); a synthetic channel at three base elevations (defect 5, continuity, uphill tolerance); runout_sim_walks on the synthetic channel; sweeps of MAX_SLOPE_DEGREES, H_L_THRESHOLD and FILL_HL_HOLES; calculate_uphill_tolerance and pcm. The rasters in Figure 12 are the ones already present in test_data/outputs. **Not done:** the GUI was read only as far as needed to describe parameters, no full-region run was repeated, and defects marked [L] (4 and 7 are exact from the code; 9 is a documentation issue) were not separately executed. The synthetic channel is a constructed test, not a calibrated case.